

Trabajo Práctico Grupal

Sistema de Aerolínea

Objetivo

Desarrollar una aplicación en Java que permita gestionar una aerolínea aplicando los conceptos de Programación Orientada a Objetos y los temas vistos en la materia.

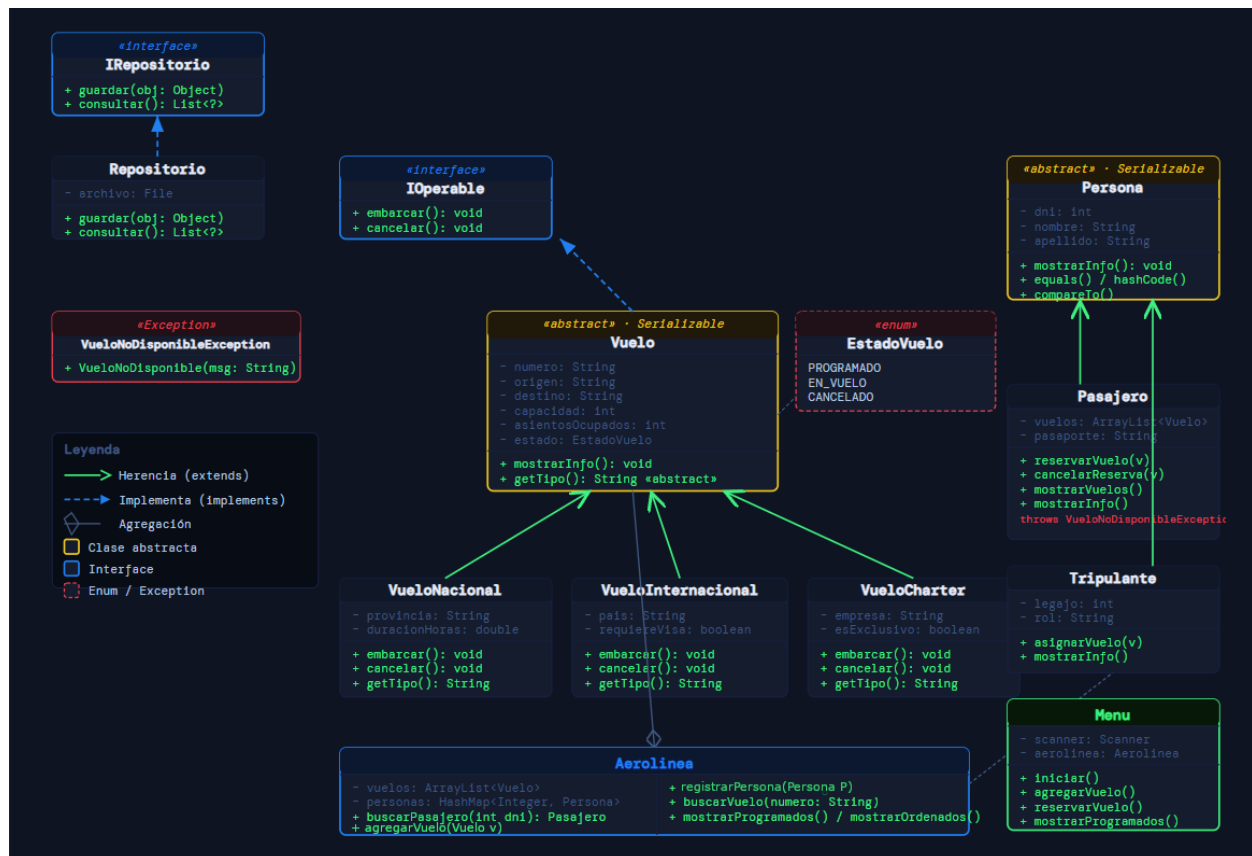
El sistema debe permitir registrar vuelos (nacional, internacional y charter), gestionar pasajeros y tripulantes, controlar reservas según el estado de cada vuelo, y persistir la información entre ejecuciones.

Modalidad

- ✓ Trabajo grupal de 3 integrantes
- _____
- ✓ 1 repositorio en GitHub por grupo
- _____
- ✓ Defensa oral obligatoria
- _____
- ✓ README con descripción, integrantes e instrucciones

💡 Cada integrante debe poder explicar **cualquier parte** del código durante la defensa. Se evaluará la comprensión individual, no solo la autoría.

Diagrama de clases



✓ Requerimientos

Modelo de dominio (POO)

- ✓ Implementar la jerarquía de herencia: Persona → Pasajero / Tripulante y Vuelo → VueloNacional / VueloInternacional / VueloCharter.
- ✓ La clase Vuelo debe ser abstracta e implementar IOperable. Definir el método abstracto getTipo() que cada subclase implementa.
- ✓ Aplicar polimorfismo: recorrer una lista de Vuelo y llamar a embarcar() y mostrarInfo() sin conocer el tipo concreto.
- ✓ Aplicar encapsulamiento: atributos privados/protegidos con getters y setters donde corresponda.
- ✓ Usar el enum EstadoVuelo (PROGRAMADO, EN_VUELO, CANCELADO) para controlar el

estado de cada vuelo.

- ---

☒ Persona implementa Comparable<Persona> (por apellido) y sobrescribe equals() y hashCode() basándose en el DNI.
-

Uso de Java Collection Framework

- ☒ Usar ArrayList<Vuelo> para los vuelos de la aerolínea y para los vuelos reservados por el pasajero.
 - ---

☒ Usar HashMap<Integer, Persona> indexado por DNI para gestionar pasajeros y tripulantes.
 - ---

☒ Usar HashSet<Persona> para evitar duplicados (por ejemplo pasajeros con al menos una reserva activa).
 - ---

☒ Usar Collections.sort() con Comparable y con un Comparator externo para ordenar vuelos por destino o número.
-

Programación funcional (Streams y Lambdas)

- ☒ Buscar vuelo por número usando Stream.filter() + findFirst().
 - ---

☒ Filtrar vuelos programados y retornar una nueva lista con collect(Collectors.toList()).
 - ---

☒ Mostrar vuelos ordenados por destino usando Stream.sorted() + lambda.
 - ---

☒ Calcular el total de asientos ocupados en todos los vuelos programados con Stream.mapToInt().sum().
 - ---

☒ Usar al menos una referencia a método (::) en un forEach.
-

Manejo de excepciones

- ✓ Crear la excepción personalizada `VueloNoDisponibleException` (checked) que se lance al intentar reservar un vuelo que está `EN_VUELO`, `CANCELADO`, o sin asientos disponibles.
 - _____
 - ✓ Manejarla en el Menu con try-catch mostrando un mensaje claro al usuario.
 - _____
 - ✓ Incluir al menos un bloque finally.
-

Serialización

- ✓ Vuelo y Persona implementan `Serializable`.
 - _____
 - ✓ Persistir la lista de vuelos en un archivo `vuelos.dat` usando `ObjectOutputStream`.
 - _____
 - ✓ Al iniciar el sistema, leer el archivo (si existe) con `ObjectInputStream` y cargar los vuelos en memoria.
 - _____
 - ✓ Usar la interfaz `IRepositorio` para encapsular la lógica de persistencia.
-

Menú por consola

El sistema debe ofrecer un menú interactivo mediante Scanner que permita como mínimo:

- ✓ Agregar vuelo (nacional, internacional o charter)
- _____
- ✓ Registrar pasajero
- _____
- ✓ Reservar vuelo
- _____
- ✓ Cancelar reserva

- ---

✓Mostrar vuelos programados
 - ---

✓Mostrar vuelos ordenados por destino
 - ---

✓Mostrar total de asientos ocupados
-

El proyecto debe compilar y ejecutarse sin errores. Incluir el archivo README.txt con nombres de los integrantes, descripción del sistema e instrucciones de ejecución.

IFES · Análisis de Sistemas · Programación 1 · TP Grupal · Sistema de Aerolínea · Java